

PL/SQL Exercise 2

Scenario 1 : Handle exceptions during fund transfers between accounts. Question: Write a stored procedure SafeTransferFunds that transfers funds between two accounts. Ensure that if any error occurs (e.g., insufficient funds), an appropriate error message is logged and the transaction is rolled back.

```
create or replace procedure SafeTransferFunds(  
    source_accId number,  
    dest_accId number,  
    transferAmt number  
)  
is  
    bal number;  
begin  
    select balance into bal from accounts where accountid = source_accId;  
  
    if bal >= transferAmt then  
        update accounts set balance = balance - transferAmt, lastmodified = sysdate  
        where accountid = source_accId;  
  
        update accounts set balance = balance + transferAmt, lastmodified = sysdate  
        where accountid = dest_accId;  
  
        commit;  
        dbms_output.put_line('Funds transferred successfully.');  
    else  
        raise_application_error(-20009, 'Insufficient Funds');  
    end if;  
  
exception  
    when no_data_found then  
        rollback;  
        dbms_output.put_line('Source account not found.');  
    when others then  
        rollback;  
        dbms_output.put_line('Error: ' || sqlerrm);  
end;  
/
```

Scenario 2 : Manage errors when updating employee salaries. Question: Write a stored procedure UpdateSalary that increases the salary of an employee by a given percentage. If the employee ID does not exist, handle the exception and log an error message.

```

create or replace procedure UpdateSalary(
    empId number,
    perInc number
)
is
    emp number;
begin
    select EmployeeID into emp from Employees
    where EmployeeID = empId;

    update Employees set Salary = Salary + (Salary * perInc / 100)
    where EmployeeID = empId;

    commit;
    dbms_output.put_line('Salary Updated');

exception
    when no_data_found then
        rollback;
        dbms_output.put_line('Employee Not Found');

    when others then
        rollback;
        dbms_output.put_line('Error : ' || sqlerrm);
end;
/

```

Scenario 3 : Ensure data integrity when adding a new customer.
 Question: Write a stored procedure AddNewCustomer that inserts a new customer into the Customers table. If a customer with the same ID already exists, handle the exception by logging an error and preventing the insertion.

```

create or replace procedure AddNewCustomer(
    custId number,
    custName varchar2,
    custDob date,
    custBalance number
)
is
begin
    insert into Customers values(custId, custName, custDob, custBalance, sysdate);

    commit;
    dbms_output.put_line('Customer Added Successfully');

exception
    when dup_val_on_index then
        rollback;

```

```
dbms_output.put_line('Customer Already Exists');
```

```
when others then  
    rollback;  
    dbms_output.put_line('Error : ' || sqlerrm);  
end;  
/
```